tf.compat.v1.feature_column.linear_model Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/feature_column/linear_model

> *Returns a linear prediction Tensor based on given feature_columns. (deprecated)*

Function Signatures

```
tf.compat.v1.feature_column.linear_model( features, feature_columns, units=1, sparse_combiner='sum', weight_collections=None, trainable=True, cols_to_vars=None )
```

Code Examples

```
tf.compat.v1.feature_column.linear_model(
features,
feature_columns,
units=1,
sparse_combiner='sum',
weight_collections=None,
trainable=True,
cols_to_vars=None
)
```

```
tf.compat.v1.feature_column.linear_model(
features,
feature_columns,
units=1,
sparse_combiner='sum',
weight_collections=None,
trainable=True,
cols_to_vars=None
)
```

```
shape = [2, 2]
{
[0, 0]: "a"
[1, 0]: "b"
[1, 1]: "c"
}
```

```
shape = [2, 2]
{
[0, 0]: "a"
[1, 0]: "b"
[1, 1]: "c"
}
```

```
price = numeric_column('price')
price_buckets = bucketized_column(price, boundaries=[0., 10., 100.,
1000.])
keywords = categorical_column_with_hash_bucket("keywords", 10K)
keywords_price = crossed_column('keywords', price_buckets, ...)
columns = [price_buckets, keywords, keywords_price ...]
features = tf.io.parse_example(...,
features=make_parse_example_spec(columns))
prediction = linear_model(features, columns)
```

```
price = numeric_column('price')
price_buckets = bucketized_column(price, boundaries=[0., 10., 100.,
1000.])
keywords = categorical_column_with_hash_bucket("keywords", 10K)
keywords_price = crossed_column('keywords', price_buckets, ...)
columns = [price_buckets, keywords, keywords_price ...]
features = tf.io.parse_example(...,
features=make_parse_example_spec(columns))
prediction = linear_model(features, columns)
```

```
# Feature 1
shape = [2, 2]
{
[0, 0]: "a"
[0, 1]: "b"
[1, 0]: "c"
}
# Feature 2
shape = [2, 3]
{
[0, 0]: "d"
[1, 0]: "e"
[1, 1]: "f"
[1, 2]: "f"
}
```

```
# Feature 1
shape = [2, 2]
{
[0, 0]: "a"
[0, 1]: "b"
[1, 0]: "c"
}
# Feature 2
shape = [2, 3]
{
[0, 0]: "d"
[1, 0]: "e"
[1, 1]: "f"
[1, 2]: "f"
}
```

Documentation

Returns a linear prediction Tensor based on given feature_columns. (deprecated)

This function generates a weighted sum based on output dimension units. Weighted sum refers to logits in classification problems. It refers to the prediction itself for linear regression problems.

Note on supported columns: linear_model treats categorical columns as indicator_columns. To be specific, assume the input as SparseTensor looks like:

linear_model assigns weights for the presence of "a", "b", "c' implicitly, just like indicator_column, while input_layer explicitly requires wrapping each of categorical columns with an embedding_column or an indicator_column.

Example of usage:

The sparse_combiner argument works as follows
For example, for two features represented as the categorical columns:

with sparse_combiner as "mean", the linear model outputs consequently are:

where y_i is the output, b is the bias, and w_x is the weight assigned to the presence of x in the input features.

- "sum": do not

normalize features in the column

- "mean": do l1 normalization on features

in the column

- "sqrtn": do l2 normalization on features in the column

Returns

Raises